<table>
<tr><td rowspan="1"></td><td colspan="2">Marwadi University
Faculty of Technology
Department of Information and Communication Technology</td></tr>
<tr><td>Subject: Artificial Intelligence (01CT0616)</td><td colspan="2">Aim: To study different word embedding for representation of textual data in vectorized form</td></tr>
<tr><td>Experiment No: 8</td><td>Date: 24-02-2026</td><td>Enrolment No: 92410133023</td></tr>
</table>

Aim: To study different word embedding for representation of textual data in vectorized form

IDE: Google Colab

Theory:

Word Embedding is an approach for representing words and documents. Word Embedding or Word Vector is a numeric vector input that represents a word in a lower-dimensional space. It allows words with similar meanings to have a similar representation.

Word Embeddings are a method of extracting features out of text so that we can input those features into a machine learning model to work with text data. They try to preserve syntactical and semantic information. The methods such as Bag of Words (BOW), CountVectorizer and TFIDF rely on the word count in a sentence but do not save any syntactical or semantic information. In these algorithms, the size of the vector is the number of elements in the vocabulary. We can get a sparse matrix if most of the elements are zero. Large input vectors will mean a huge number of weights which will result in high computation required for training. Word Embeddings give a solution to these problems.

Need for Word Embedding?

- To reduce dimensionality
- To use a word to predict the words around it.
- Inter-word semantics must be captured.

How are Word Embeddings used?

- They are used as input to machine learning models. Take the words —-> Give their numeric representation —-> Use in training or inference.
- To represent or visualize any underlying patterns of usage in the corpus that was used to train them.

1. Bag of Words

Bag of Words (BOW) is an algorithm that counts how many times a word appears in a document. Those word counts allow us to compare documents and gauge their similarities for applications like search, document classification, and topic modeling.

2. Tf-Idf

Tf-Idf is shorthand for term frequency-inverse document frequency. So, two things: term frequency and inverse document frequency. Term frequency (TF) is basically the output of the BoW model. For a specific document, it determines how important a word is by looking at how frequently it appears in the document. Term frequency measures the importance of the word. If a word appears a lot of times, then the word must be important. For example, if our document is "I am a cat lover. I have a cat named Steve. I feed a cat outside my room regularly," we see that the words with the highest frequency are I, a, and cat. This agrees with our intuition that high term frequency = higher importance.

TF(t) = (Number of times term t appears in a document) / (Total number of terms in the document)

IDF used to calculate the weight of rare words across all documents. The words that occur rarely in the corpus have a high IDF score. However, it is known that certain terms, such as “I”, “a” may appear a lot of times but have little importance. Thus we need to weigh down the frequent terms while scaling up the rare ones

IDF(t) = log_e(Total number of documents / Number of documents with term t in it)

Pre Lab Exercise:

1. Perform BoW vectorization of the corpus

D1: "I love AI"
D2: "AI is the future"
D3: "I love the future"

[I, love, AI, is, the, future]

Step 2: Create Vectors (Count of words)

Document	**I**	**love**	**AI**	**is**	**the**	**future**
2.D1	1	1	1	0	0	0
3.D2	0	0	1	1	1	1
4.D3	1	1	0	0	1	1

2.

This is **BoW** → simple word count.

2. Perform TF vectorization of the corpus

TF = (Number of times word appears in document) / (Total words in document)

Step 1: Count total words

- D1 → 3 words
- D2 → 4 words
- D3 → 4 words

Step 2: Compute TF

Document	**I**	**love**	**AI**	**is**	**the**	**future**
D1	**1/3**	**1/3**	**1/3**	**0**	**0**	**0**
D2	**0**	**0**	**1/4**	**1/4**	**1/4**	**1/4**
D3	**1/4**	**1/4**	**0**	**0**	**1/4**	**1/4**

This is TF vectorization → normalized frequency.

Program (Code):

To be attached with

1. Perform the TF-IDF vectorization of the corpus

```
# @title
sentences= [
"I am living in the india country.",
"India is my country all Indians are my brothers and sisters except one girl.",
"In my country we follow the snathana dharmam where we respect our tradition and culture",

"I like to play cricketin my heart",
"eleven members can paly the cricket",

"Now a days the india is becoming the semiconductor hub",
"we will make our own chips in our country like we have made the virkram 32-bit chip." ]
# @title
#import the libraries
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer #TF-BOW
from sklearn.feature_extraction.text import TfidfVectorizer #TF-IDF
from sklearn.metrics.pairwise import cosine_similarity
# @title
#TF-BOW
tf_vectorizer=CountVectorizer()
tf_matrix=tf_vectorizer.fit_transform(sentences)
# @title
tf_matrix
# @title
tf_similarity=cosine_similarity(tf_matrix)
print(tf_similarity)
```

```
# @title
##TF-IDF embedding
tfidf_vectorizer=TfidfVectorizer()
tfidf_matrix=tfidf_vectorizer.fit_transform(sentences)
# @title
tfidf_similarity=cosine_similarity(tfidf_matrix)
# @title
tfidf_matrix
# @title
print(tfidf_similarity)
# @title
#Document similarity function
def doc_sim(similarity_matrix,sentences):
  for i in range(len(sentences)):
    sim =similarity_matrix[i].copy()
    sim[i]=-1
    top_index=np.argmax(sim)
    print(sentences[i],"-->",sentences[top_index])

# @title
doc_sim(tfidf_similarity,sentences)
```

Results:

To be attached with

I am living in the india country. --> Now a days the india is becoming the semiconductor hub
India is my country all Indians are my brothers and sisters except one girl. --> In my country we follow the snathana dharmam where we respect our tradition and culture
In my country we follow the snathana dharmam where we respect our tradition and culture --> we will make our own chips in our country like we have made the virkram 32-bit chip.
I like to play cricketin my heart --> India is my country all Indians are my brothers and sisters except one girl.
eleven members can paly the cricket --> Now a days the india is becoming the semiconductor hub
Now a days the india is becoming the semiconductor hub --> I am living in the india country.
we will make our own chips in our country like we have made the virkram 32-bit chip. --> In my country we follow the snathana dharmam where we respect our tradition and culture

Observation:

Post Lab Exercise:

Take three documents and find similarity using

D1: I love AI
D2: AI is the future
D3: I love the future

[I, love, AI, is, the, future]

a. BoW vectorization

Document	I	love	AI	is	the	future
D1	1	1	1	0	0	0
D2	0	0	1	1	1	1
D3	1	1	0	0	1	1

Pair	Similarity
D1 & D2	0.33
D1 & D3	0.67
D2 & D3	0.67

b. TF Vectorization

Term Frequency (TF)

TF = (Word Count in Document) / (Total Words in Document)

Document	I	love	AI	is	the	future
D1	1/3	1/3	1/3	0	0	0
D2	0	0	1/4	1/4	1/4	1/4
D3	1/4	1/4	0	0	1/4	1/4

Pair	**Similarity**
D1 & D2	0.29
D1 & D3	0.58
D2 & D3	0.75

c. TF-IDF Vectorization

TF-IDF reduces the importance of common words and increases the importance of rare words.

Document	**I**	**love**	**AI**	**is**	**the**	**future**
D1	0.58	0.58	0.58	0	0	0
D2	0	0	0.40	0.69	0.51	0.51
D3	0.51	0.51	0	0	0.51	0.51

Pair	**Similarity**
D1 & D2	0.23
D1 & D3	0.52
D2 & D3	0.66

Comment over the answer obtained in each of the case.

In the case of Bag of Words (BoW), the similarity is calculated purely on the basis of the frequency of words in each document. It does not consider the importance of words or the length of the document. As a result, documents D1 and D3 show the highest similarity because they share common words such as “I” and “love”. However, this method treats all words equally, including very common words, which may not carry significant meaning. Therefore, although BoW is simple and easy to implement, it does not always provide accurate or meaningful similarity results.

In the Term Frequency (TF) approach, the word counts are normalized by dividing them by the total number of words in each document. This helps in reducing the bias caused by different document lengths. In this case, documents D2 and D3 show the highest similarity because they share words like “the” and “future”. Compared to BoW, TF gives more balanced results. However, it still assigns equal importance to all words, including common words, which can affect the accuracy of similarity measurement.

In the TF-IDF approach, both the frequency of words and their importance across all documents are considered.

Common words such as "the" are given lower weight, while rare and meaningful words are given higher importance. This results in a more accurate and realistic similarity measure. In this experiment, documents D2 and D3 again show the highest similarity, but the values are more refined compared to TF. TF-IDF successfully reduces the impact of less meaningful words and highlights the significance of important terms.